

```
<?php

// manage_traceability.php

// Traceability Oversight - single-file admin module

// - View QR codes (batches)

// - View supply chain logs

// - Detailed timeline (cards) with actor role + actor full name

// - AJAX handlers (fetch batches, fetch logs, add log, delete log, suggestions)

// Save as admin/manage_traceability.php


session_start();


// ----- CONFIG -----

$DB_HOST = '127.0.0.1';

$DB_NAME = 'agriculture_db'; // change if needed

$DB_USER = 'root';

$DB_PASS = '';


// require admin logged in

if (!isset($_SESSION['admin'])) {

    header("Location: index.php");

    exit;

}


// ----- PDO Connection -----

try {

    $pdo = new PDO(
```

```

        "mysql:host={$DB_HOST};dbname={$DB_NAME};charset=utf8mb4",
        $DB_USER,
        $DB_PASS,
        [PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION]
    );
} catch (Exception $e) {
    echo "DB connection error: " . htmlspecialchars($e->getMessage());
    exit;
}

// helper: check if table exists
function tableExists(PDO $pdo, $table) {
    $stmt = $pdo->prepare("SELECT COUNT(*) FROM information_schema.tables WHERE
table_schema = :db AND table_name = :table");

    $stmt->execute([':db' => $pdo->query('select database()')->fetchColumn(), ':table' =>
$table]);

    return (bool)$stmt->fetchColumn();
}

$has_users = tableExists($pdo, 'users');

// ----- AJAX HANDLER -----
if ($_SERVER['REQUEST_METHOD'] === 'POST' && isset($_POST['action'])) {
    header('Content-Type: application/json; charset=utf-8');

    $action = $_POST['action'];

    try {
        // 1) fetch_batches - paginated list of crop_batches with joined crop & farmer
        if ($action === 'fetch_batches') {

```

```

$page = max(1, (int)($_POST['page'] ?? 1));

$limit = (int)($_POST['limit'] ?? 10);

$start = ($page - 1) * $limit;

$filter = trim($_POST['filter'] ?? "");

if ($filter !== "") {

    $like = "%$filter%";

    $stmt = $pdo->prepare("SELECT b.*, c.crop_type, c.variety, c.farmer_id,
f.first_name, f.middle_name, f.last_name

        FROM crop_batches b

        LEFT JOIN crops c ON c.id = b.crop_id

        LEFT JOIN farmers f ON f.id = c.farmer_id

        WHERE b.batch_code LIKE :f OR c.crop_type LIKE :f OR f.first_name LIKE :f OR
f.last_name LIKE :f

        ORDER BY b.id DESC LIMIT :s, :l");

    $stmt->bindValue(':f', $like, PDO::PARAM_STR);

    $stmt->bindValue(':s', $start, PDO::PARAM_INT);

    $stmt->bindValue(':l', $limit, PDO::PARAM_INT);

    $stmt->execute();

    $rows = $stmt->fetchAll(PDO::FETCH_ASSOC);

    $countStmt = $pdo->prepare("SELECT COUNT(*) FROM crop_batches b LEFT JOIN
crops c ON c.id = b.crop_id LEFT JOIN farmers f ON f.id = c.farmer_id WHERE b.batch_code
LIKE :f OR c.crop_type LIKE :f OR f.first_name LIKE :f OR f.last_name LIKE :f");

    $countStmt->execute(['f' => $like]);

    $total = (int)$countStmt->fetchColumn();

} else {

```

```

        $stmt = $pdo->prepare("SELECT b.*, c.crop_type, c.variety, c.farmer_id,
f.first_name, f.middle_name, f.last_name

        FROM crop_batches b

        LEFT JOIN crops c ON c.id = b.crop_id

        LEFT JOIN farmers f ON f.id = c.farmer_id

        ORDER BY b.id DESC LIMIT :s, :l");

        $stmt->bindValue(':s', $start, PDO::PARAM_INT);

        $stmt->bindValue(':l', $limit, PDO::PARAM_INT);

        $stmt->execute();

        $rows = $stmt->fetchAll(PDO::FETCH_ASSOC);


        $total = (int)$pdo->query("SELECT COUNT(*) FROM crop_batches")-
>fetchColumn();

    }


    // attach farmer full name and QR URL fallback

    foreach ($rows as &$r) {

        $r['farmer_name'] = trim(($r['first_name'] ?? " ") . ' ' . ($r['middle_name'] ?? " ") . ' ' .
($r['last_name'] ?? " "));

        if (empty($r['qr_code_path'])) {

            // optionally generate QR data URI? For now, leave blank and the frontend will
            show placeholder.

            $r['qr_code_path'] = "";

        }

    }

    echo json_encode(['success' => true, 'data' => $rows, 'total' => $total, 'page' =>
$page, 'limit' => $limit]);

```

```
exit;  
}
```

```
// 2) fetch_logs - paginated logs for supply chain table (can filter by actor_role)
```

```
if ($action === 'fetch_logs') {
```

```
    $page = max(1, (int)($_POST['page'] ?? 1));
```

```
    $limit = (int)($_POST['limit'] ?? 12);
```

```
    $start = ($page - 1) * $limit;
```

```
    $role = trim($_POST['role'] ?? "");
```

```
    $filter = trim($_POST['filter'] ?? "");
```

```
    $where = [];
```

```
    $params = [];
```

```
    if ($role !== "") { $where[] = "actor_role = :role"; $params[':role'] = $role; }
```

```
    if ($filter !== "") { $where[] = "(notes LIKE :f OR status LIKE :f)"; $params[':f'] =  
"%$filter%"; }
```

```
    $sql = "SELECT l.*, b.batch_code, c.crop_type, f.first_name, f.middle_name,  
f.last_name
```

```
    FROM batch_status_logs l
```

```
    LEFT JOIN crop_batches b ON b.id = l.batch_id
```

```
    LEFT JOIN crops c ON c.id = b.crop_id
```

```
    LEFT JOIN farmers f ON f.id = c.farmer_id";
```

```
    if (!empty($where)) $sql .= " WHERE " . implode(' AND ', $where);
```

```
    $sql .= " ORDER BY l.timestamp DESC LIMIT :s, :l";
```

```
    $stmt = $pdo->prepare($sql);
```

```
    foreach ($params as $k => $v) $stmt->bindValue($k, $v);
```

```

$stmt->bindValue(':s', $start, PDO::PARAM_INT);

$stmt->bindValue(':l', $limit, PDO::PARAM_INT);

$stmt->execute();

$rows = $stmt->fetchAll(PDO::FETCH_ASSOC);


// total

if (!empty($where)) {

    $countSql = "SELECT COUNT(*) FROM batch_status_logs l LEFT JOIN crop_batches
b ON b.id = l.batch_id LEFT JOIN crops c ON c.id = b.crop_id LEFT JOIN farmers f ON f.id =
c.farmer_id WHERE " . implode(' AND ', $where);

    $countStmt = $pdo->prepare($countSql);

    foreach ($params as $k => $v) $countStmt->bindValue($k, $v);

    $countStmt->execute();

    $total = (int)$countStmt->fetchColumn();

} else {

    $total = (int)$pdo->query("SELECT COUNT(*) FROM batch_status_logs")-
>fetchColumn();

}


// attach actor_name

foreach ($rows as &$r) {

    $r['farmer_name'] = trim(($r['first_name'] ?? '') . ' ' . ($r['middle_name'] ?? '') . ' ' .
($r['last_name'] ?? ''));

    $r['actor_full_name'] = "";

    if ($r['actor_role'] === 'farmer') {

        // get farmer name for actor_id if differs

        $aid = (int)$r['actor_id'];

```

```

        if ($aid > 0) {
            $st = $pdo->prepare("SELECT first_name, middle_name, last_name FROM
farmers WHERE id = :id");

            $st->execute([':id' => $aid]);

            $ff = $st->fetch(PDO::FETCH_ASSOC);

            if ($ff) $r['actor_full_name'] = trim($ff['first_name']. ' '.$ff['middle_name'].
'.'.$ff['last_name']);
        }
    } else {
        if ($has_users) {
            $st = $pdo->prepare("SELECT full_name FROM users WHERE id = :id");

            $st->execute([':id' => (int)$r['actor_id']]);

            $ux = $st->fetchColumn();

            if ($ux) $r['actor_full_name'] = $ux;
        }
    }
}

echo json_encode(['success' => true, 'data' => $rows, 'total' => $total, 'page' =>
$page, 'limit' => $limit]);

exit;
}

```

// 3) fetch\_timeline - all logs for a batch (timeline cards)

```

if ($action === 'fetch_timeline') {
    $batch_id = (int)($_POST['batch_id'] ?? 0);

    if ($batch_id <= 0) throw new Exception('Invalid batch id');
}

```

```

$stmt = $pdo->prepare("SELECT l.*, b.batch_code, c.crop_type, c.variety, f.first_name,
f.middle_name, f.last_name

FROM batch_status_logs l

LEFT JOIN crop_batches b ON b.id = l.batch_id

LEFT JOIN crops c ON c.id = b.crop_id

LEFT JOIN farmers f ON f.id = c.farmer_id

WHERE l.batch_id = :bid

ORDER BY l.timestamp ASC");

$stmt->execute([':bid' => $batch_id]);

$rows = $stmt->fetchAll(PDO::FETCH_ASSOC);

// attach actor full name where possible
foreach ($rows as &$r) {

    $r['farmer_name'] = trim(($r['first_name'] ?? '') . ' ' . ($r['middle_name'] ?? '') . ' ' .
($r['last_name'] ?? ''));

    $r['actor_full_name'] = "";

    if ($r['actor_role'] === 'farmer') {

        $aid = (int)$r['actor_id'];

        if ($aid > 0) {

            $st = $pdo->prepare("SELECT first_name, middle_name, last_name FROM
farmers WHERE id = :id");

            $st->execute([':id' => $aid]);

            $ff = $st->fetch(PDO::FETCH_ASSOC);

            if ($ff) $r['actor_full_name'] = trim($ff['first_name'] . ' ' . $ff['middle_name'] .
' ' . $ff['last_name']);

        }

    } else {

```

```

        if ($has_users) {
            $st = $pdo->prepare("SELECT full_name FROM users WHERE id = :id");
            $st->execute([':id'=>(int)$r['actor_id']);
            $ux = $st->fetchColumn();
            if ($ux) $r['actor_full_name'] = $ux;
        }
    }
}

echo json_encode(['success' => true, 'data' => $rows]);
exit;
}

// 4) add_log - insert new status log for a batch
if ($action === 'add_log') {
    $batch_id = (int)($_POST['batch_id'] ?? 0);
    $actor_role = trim($_POST['actor_role'] ?? '');
    $actor_id = (int)($_POST['actor_id'] ?? 0);
    $status = trim($_POST['status'] ?? '');
    $notes = trim($_POST['notes'] ?? '');
    $location = trim($_POST['location'] ?? '');
    $timestamp = trim($_POST['timestamp'] ?? date('Y-m-d H:i:s'));

    if ($batch_id <= 0 || $actor_role === '' || $status === '') throw new Exception('Batch,
    actor role and status are required');
}

```

```
$stmt = $pdo->prepare("INSERT INTO batch_status_logs (batch_id, actor_role, actor_id, status, notes, location, timestamp) VALUES (:bid, :ar, :aid, :st, :notes, :loc, :ts)");
```

```
$stmt->execute([  
    ':bid' => $batch_id,  
    ':ar' => $actor_role,  
    ':aid' => $actor_id > 0 ? $actor_id : null,  
    ':st' => $status,  
    ':notes' => $notes,  
    ':loc' => $location,  
    ':ts' => $timestamp  
]);
```

```
echo json_encode(['success' => true, 'message' => 'Log added']);
```

```
exit;
```

```
}
```

```
// 5) delete_log
```

```
if ($action === 'delete_log') {
```

```
    $id = (int)($_POST['id'] ?? 0);
```

```
    if ($id <= 0) throw new Exception('Invalid id');
```

```
    $stmt = $pdo->prepare("DELETE FROM batch_status_logs WHERE id = :id");
```

```
    $stmt->execute([':id' => $id]);
```

```
    echo json_encode(['success' => true, 'message' => 'Log deleted']);
```

```
    exit;
```

```
}
```

```

// 6) batch_suggestions (autocomplete)

if ($action === 'batch_suggestions') {

    $q = trim($_POST['q'] ?? '');

    $stmt = $pdo->prepare("SELECT b.id, b.batch_code, c.crop_type, f.first_name,
f.middle_name, f.last_name

        FROM crop_batches b

        LEFT JOIN crops c ON c.id = b.crop_id

        LEFT JOIN farmers f ON f.id = c.farmer_id

        WHERE b.batch_code LIKE :q OR c.crop_type LIKE :q OR f.first_name LIKE :q OR
f.last_name LIKE :q

        ORDER BY b.created_at DESC LIMIT 8");

    $stmt->execute([':q' => "%$q%"]);

    $rows = $stmt->fetchAll(PDO::FETCH_ASSOC);

    foreach ($rows as &$r) {

        $r['farmer_name'] = trim(($r['first_name'] ?? '') . ' ' . ($r['middle_name'] ?? '') . ' ' .
($r['last_name'] ?? ''));

    }

    echo json_encode(['success' => true, 'data' => $rows]);

    exit;

}

} catch (Exception $e) {

    echo json_encode(['success' => false, 'message' => $e->getMessage()]);

    exit;

}

echo json_encode(['success' => false, 'message' => 'Unknown action']);

exit;

```

```
}
```

```
// ----- PAGE (GET) -----
```

```
$cntBatches = (int)$pdo->query("SELECT COUNT(*) FROM crop_batches")->fetchColumn();
```

```
$cntLogs = (int)$pdo->query("SELECT COUNT(*) FROM batch_status_logs")->fetchColumn();
```

```
$cntCrops = (int)$pdo->query("SELECT COUNT(*) FROM crops")->fetchColumn();
```

```
?>
```

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
<meta charset="utf-8">
```

```
<title>Traceability Oversight — Admin</title>
```

```
<meta name="viewport" content="width=device-width,initial-scale=1">
```

```
<link
```

```
href="https://fonts.googleapis.com/css2?family=Poppins:wght@300;400;500;600&display=swap" rel="stylesheet">
```

```
<link href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.5.0/css/all.min.css" rel="stylesheet">
```

```
<style>
```

```
:root{
```

```
--bg:#f5f7fb;--card:#fff;--primary:#0b74de;--muted:#6b7a86;--shadow:0 10px 30px rgba(10,20,30,0.06);
```

```
}
```

```
*{box-sizing:border-box}
```

```
body{margin:0;font-family:'Poppins',sans-serif;background:var(--bg);color:#111}
```

```
.sidebar{position:fixed;left:0;top:0;width:220px;height:100vh;background:#0f1724;color:#fff;padding:20px;box-shadow:2px 0 20px rgba(2,6,23,0.25)}
```

```
.brand{font-weight:700;margin-bottom:18px;display:flex;gap:8px;align-items:center}
```

```
.side-nav a{display:flex;align-items:center;gap:10px;padding:10px;border-radius:8px;color:#cfe0ff;text-decoration:none;margin-bottom:6px}
```

```
.side-nav a:hover{background:rgba(255,255,255,0.03)}
```

```
.main{margin-left:240px;padding:20px}
```

```
.header{display:flex;justify-content:space-between;align-items:center;gap:12px}
```

```
.cards{display:flex;gap:14px;margin-top:16px}
```

```
.card{background:var(--card);padding:12px;border-radius:12px;box-shadow:var(--shadow);flex:1}
```

```
.search-row{display:flex;gap:12px;align-items:center;margin-top:14px}
```

```
.search-box{position:relative;width:420px}
```

```
.search-box input{width:100%;padding:10px;border-radius:8px;border:1px solid #e6eef8}
```

```
.suggestions{position:absolute;left:0;right:0;top:40px;background:#fff;border-radius:8px;box-shadow:0 10px 30px rgba(10,20,30,0.08);display:none;z-index:40}
```

```
.suggestions p{margin:0;padding:10px;border-bottom:1px solid #f1f4f8;cursor:pointer}
```

```
.suggestions p:hover{background:#f7fbff}
```

```
/* batches table */
```

```
.table{width:100%;border-collapse:collapse;margin-top:12px;background:var(--card);border-radius:10px;box-shadow:var(--shadow);overflow:hidden}
```

```
.table th, .table td{padding:12px;text-align:left;border-bottom:1px solid #f1f4f8}
```

```
.table thead th{background:#f7f9fc;color:#555;font-weight:600}
```

```
.qr-thumb{width:72px;height:72px;border-radius:8px;display:inline-flex;align-items:center;justify-content:center;background:#fbfbfb;border:1px dashed #e6eef8}
```

```
.btn{padding:8px 10px;border-radius:8px;border:none;cursor:pointer}
```

```
.btn-primary{background:var(--primary);color:#fff}
```

```
.btn-soft{background:#eef6ff;color:var(--primary)}

/* timeline cards (detailed) */

.timeline{
  display:flex;flex-direction:column;gap:14px;margin-top:14px;
}

.card-t{background:var(--card);border-radius:12px;padding:14px;box-shadow:var(--shadow)}

.card-t .meta{display:flex;justify-content:space-between;align-items:center;gap:8px}

.role-badge{display:inline-block;padding:6px 10px;border-radius:999px;font-weight:600;font-size:13px}

.role-farmer{background:#e8f8f1;color:#116644}
.role-collector{background:#fff4e6;color:#a75b00}
.role-processor{background:#eef3ff;color:#123a8a}
.role-supplier{background:#fff0f6;color:#7a2a4a}
.role-export{background:#fffbe6;color:#8a6b00}

/* small */

.small-muted{color:var(--muted);font-size:13px}

.actions{display:flex;gap:8px}

/* responsive */

@media (max-width:900px){.sidebar{display:none}.main{margin-left:12px;padding:12px}.search-box{width:100%}}

</style>

</head>

<body>
```

```
<div class="sidebar">

  <div class="brand"><i class="fa fa-seedling"></i> AGRO-TRACE</div>

  <div class="side-nav">

    <a href="dashboard.php"><i class="fa fa-chart-line"></i> Dashboard</a>

    <a href="manage_farmers.php"><i class="fa fa-users"></i> Farmers</a>

    <a href="manage_crops.php"><i class="fa fa-leaf"></i> Crops</a>

    <a href="manage_advisories.php"><i class="fa fa-bullhorn"></i> Advisories</a>

    <a href="#" style="background:rgba(255,255,255,0.03)"><i class="fa fa-qrcode"></i>
Traceability</a>

    <a href="logout.php"><i class="fa fa-power-off"></i> Logout</a>

  </div>

</div>
```

```
<div class="main">

  <div class="header">

    <div>

      <h2 style="margin:0">Traceability Oversight</h2>

      <div class="small-muted">View QR codes, supply chain logs and detailed product
timelines</div>

    </div>

    <div style="display:flex;gap:8px;align-items:center">

      <div class="search-box">

        <input id="batchSearch" placeholder="Search batch code, crop or farmer...">

        <div id="batchSuggestions" class="suggestions"></div>

      </div>

    </div>

  </div>
```

```
<button id="refreshBtn" class="btn btn-soft"><i class="fa fa-arrows-rotate"></i></button>
```

```
</div>
```

```
</div>
```

```
<div class="cards">
```

```
<div class="card"><div class="small-muted">Total  
Batches</div><h3><?=$cntBatches?></h3></div>
```

```
<div class="card"><div class="small-muted">Total  
Logs</div><h3><?=$cntLogs?></h3></div>
```

```
<div class="card"><div class="small-muted">Crop  
Types</div><h3><?=$cntCrops?></h3></div>
```

```
</div>
```

```
<div style="margin-top:18px">
```

```
<!-- Batches table -->
```

```
<div id="batchesWrap"></div>
```

```
<div id="batchesPager" style="margin-top:12px;text-align:center"></div>
```

```
</div>
```

```
<hr style="margin:18px 0">
```

```
<div style="display:flex;gap:18px;align-items:flex-start">
```

```
<!-- Left: supply chain logs table -->
```

```
<div style="flex:1">
```

```
<div style="display:flex;justify-content:space-between;align-items:center">
```

```
<h3 style="margin:0">Supply Chain Logs</h3>
```

```
<div style="display:flex;gap:8px;align-items:center">
  <select id="filterRole" style="padding:8px;border-radius:8px;border:1px solid
#e6eef8">
    <option value="">All roles</option>
    <option value="farmer">Farmer</option>
    <option value="collector">Collector</option>
    <option value="processor">Processor</option>
    <option value="supplier">Supplier</option>
    <option value="export">Export</option>
  </select>
  <input id="logsFilter" placeholder="Filter notes or status" style="padding:8px;border-
radius:8px;border:1px solid #e6eef8">
  <button id="applyLogsFilter" class="btn btn-primary">Apply</button>
</div>
</div>
```

```
<div id="logsWrap" style="margin-top:12px"></div>
<div id="logsPager" style="margin-top:12px;text-align:center"></div>
</div>
```

<!-- Right: timeline viewer -->

```
<div style="width:480px">
  <div style="display:flex;justify-content:space-between;align-items:center">
    <h3 style="margin:0">Product Timeline</h3>
    <div class="small-muted">Select a batch → View timeline</div>
  </div>
```

```

<div id="timelineWrap" class="timeline" style="margin-top:12px">

  <div class="small-muted">No batch selected. Click "View Timeline" on a batch row to
load details.</div>

</div>

<!-- Add log modal (inline panel) -->

<div style="margin-top:12px;background:var(--card);padding:12px;border-
radius:10px;box-shadow:var(--shadow)">

  <h4 style="margin:0 0 8px 0">Add Movement Log</h4>

  <form id="addLogForm">

    <input type="hidden" name="batch_id" id="log_batch_id">

    <div style="display:flex;gap:8px;margin-bottom:8px">

      <select name="actor_role" id="log_actor_role" style="flex:1;padding:8px;border-
radius:8px;border:1px solid #e6eef8">

        <option value="farmer">Farmer</option>

        <option value="collector">Collector</option>

        <option value="processor">Processor</option>

        <option value="supplier">Supplier</option>

        <option value="export">Export</option>

      </select>

      <input name="actor_id" id="log_actor_id" placeholder="Actor ID (optional)"
style="width:140px;padding:8px;border-radius:8px;border:1px solid #e6eef8">

    </div>

    <input name="status" id="log_status" placeholder="Status (e.g., Delivered, Received,
Quality check passed)" required style="width:100%;padding:8px;border-
radius:8px;border:1px solid #e6eef8;margin-bottom:8px">

```

```
<input name="location" id="log_location" placeholder="Location (optional)"
style="width:100%;padding:8px;border-radius:8px;border:1px solid #e6eef8;margin-
bottom:8px">
```

```
<textarea name="notes" id="log_notes" placeholder="Notes (optional)" rows="2"
style="width:100%;padding:8px;border-radius:8px;border:1px solid #e6eef8"></textarea>
```

```
<div style="display:flex;gap:8px;justify-content:flex-end;margin-top:8px">
```

```
<button type="submit" class="btn btn-primary">Add Log</button>
```

```
</div>
```

```
</form>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
<div id="toast"
style="position:fixed;right:20px;bottom:20px;background:#111;color:#fff;padding:12px;bo
rder-radius:8px;display:none;z-index:999"></div>
```

```
<script>
```

```
// ----- Utilities -----
```

```
function qs(s){return document.querySelector(s);}
```

```
function qsa(s){return Array.from(document.querySelectorAll(s));}
```

```
function showToast(msg, ok=true){ const t=qs('#toast'); t.style.background = ok? '#111' :
'#b71c1c'; t.textContent = msg; t.style.display='block'; setTimeout(()=>
t.style.display='none', 3000); }
```

```
function escapeHtml(s){return String(s||'').replace(/[\&<>'\"\\]/g,function(ch){return  
{'&':'&amp;','<':'&lt;','>':'&gt;','\"':'&quot;','\\':'&#39;','/':'&#x2F;'}[ch];});}  
  
function formatDateTime(s){ if(!s) return ''; const d=new Date(s); return d.toLocaleString(); }
```

```
// ----- State -----
```

```
let batchesPage = 1, batchesLimit = 8;
```

```
let logsPage = 1, logsLimit = 8;
```

```
let selectedBatchId = null;
```

```
// ----- Load batches -----
```

```
function loadBatches(page=1, filter='') {  
  batchesPage = page;  
  const fd = new FormData();  
  fd.append('action','fetch_batches');  
  fd.append('page', page);  
  fd.append('limit', batchesLimit);  
  fd.append('filter', filter);  
  fetch(location.pathname, {method:'POST', body: fd}).then(r=>r.json()).then(res=>{  
    if(!res.success) return showToast(res.message||'Failed to load batches', false);  
    renderBatches(res.data);  
    renderPager('batchesPager', res.total, res.page, res.limit, loadBatches);  
  }).catch(()=> showToast('Server error', false));  
}
```

```
function renderBatches(rows) {  
  const wrap = qs('#batchesWrap');
```

```
if(!rows || rows.length===0) { wrap.innerHTML = '<div style="padding:14px;color:#777">No  
batches found.</div>'; return; }
```

```
let html = `<table  
class="table"><thead><tr><th>QR</th><th>Batch</th><th>Crop</th><th>Farmer</th><th>  
>Qty</th><th>Harvest</th><th>Actions</th></tr></thead><tbody>`;
```

```
rows.forEach(r=>{  
    const farmer = escapeHtml(r.farmer_name || '—');  
    const qr = r.qr_code_path ? `` : `<div class="qr-thumb"><i class="fa fa-qrcode"></i></div>`;
```

```
html += `<tr>  
    <td>${qr}</td>  
    <td>${escapeHtml(r.batch_code)}</td>  
    <td>${escapeHtml(r.crop_type||'—')} ${r.variety? ('/'+escapeHtml(r.variety)): ''}</td>  
    <td>${farmer}</td>  
    <td>${escapeHtml(r.quantity||'—')}</td>  
    <td>${escapeHtml(r.harvest_date||'—')}</td>  
    <td>  
        <div class="actions">  
            <button class="btn btn-soft" onclick="viewTimeline(${r.id})"><i class="fa fa-clock-  
rotate-left"></i> View Timeline</button>  
            <button class="btn btn-primary" onclick="selectBatchForLog(${r.id},  
'${escapeHtml(r.batch_code)}')"><i class="fa fa-plus"></i> Add Log</button>
```

```
        </div>  
    </td>  
</tr>`;  
});  
html += `</tbody></table>`;
```

```
wrap.innerHTML = html;
```

```
}
```

```
// ----- Pager renderer -----
```

```
function renderPager(containerId, total, page, limit, onClickFn) {
```

```
    const container = qs('#' + containerId);
```

```
    container.innerHTML = "";
```

```
    const pages = Math.max(1, Math.ceil(total/limit));
```

```
    for (let i=1;i<=pages;i++){
```

```
        const b = document.createElement('button');
```

```
        b.textContent = i;
```

```
        b.className = 'btn';
```

```
        if (i===page) { b.style.background = '#0b74de'; b.style.color = '#fff'; }
```

```
        b.addEventListener('click', ()=> onClickFn(i));
```

```
        container.appendChild(b);
```

```
    }
```

```
}
```

```
// ----- Load logs -----
```

```
function loadLogs(page=1) {
```

```
    logsPage = page;
```

```
    const fd = new FormData();
```

```
    fd.append('action','fetch_logs');
```

```
    fd.append('page', page);
```

```
    fd.append('limit', logsLimit);
```

```
    fd.append('role', qs('#filterRole').value || "");
```

```
    fd.append('filter', qs('#logsFilter').value.trim() || "");
```

```

fetch(location.pathname, {method:'POST', body: fd}).then(r=>r.json()).then(res=>{
  if(!res.success) return showToast(res.message||'Failed to load logs', false);
  renderLogs(res.data);
  renderPager('logsPager', res.total, res.page, res.limit, loadLogs);
}).catch(()=> showToast('Server error', false));
}

```

```

function renderLogs(rows) {
  const wrap = qs('#logsWrap');

  if(!rows || rows.length===0){ wrap.innerHTML = '<div style="padding:12px;color:#777">No
logs found.</div>'; return; }

  let html = `<table
class="table"><thead><tr><th>Batch</th><th>Actor</th><th>Status</th><th>Notes</th>
<th>Time</th><th>Action</th></tr></thead><tbody>`;

  rows.forEach(r=>{

    const batch = escapeHtml(r.batch_code || '—');

    const actorRole = escapeHtml(r.actor_role || '');

    const actorName = escapeHtml(r.actor_full_name || (r.actor_id ? (actorRole + ' #' +
r.actor_id) : '—'));

    const badgeClass = 'role-badg' + (r.actor_role === 'farmer' ? 'role-farmer' : (r.actor_role
=== 'collector' ? 'role-collector' : (r.actor_role === 'processor' ? 'role-processor' :
(r.actor_role==='supplier'? 'role-supplier': 'role-export'))));

    html += `<tr>

    <td>${batch}</td>

    <td><span class="${badgeClass}">${actorRole}</span><div style="font-
size:13px;margin-top:6px">${actorName}</div></td>

    <td>${escapeHtml(r.status)}</td>

    <td>${escapeHtml(r.notes||'—')}</td>

```

```

        <td>${escapeHtml(r.timestamp)}</td>

        <td><button class="btn btn-soft" onclick="deleteLog(${r.id})"><i class="fa fa-
trash"></i></button></td>

    </tr>`;

});

html += `</tbody></table>`;

wrap.innerHTML = html;
}

// ----- View timeline (detailed cards) -----
function viewTimeline(batchId) {
    selectedBatchId = batchId;
    const fd = new FormData();
    fd.append('action','fetch_timeline');
    fd.append('batch_id', batchId);
    fetch(location.pathname, {method:'POST', body: fd}).then(r=>r.json()).then(res=>{
        if(!res.success) return showToast(res.message||'Failed to load timeline', false);
        renderTimeline(res.data);
        // also scroll to timeline
        window.scrollTo({top: document.body.scrollHeight, behavior: 'smooth'});
    }).catch(()=> showToast('Server error', false));
}

function renderTimeline(rows) {
    const wrap = qs('#timelineWrap');

    if(!rows || rows.length===0) { wrap.innerHTML = '<div class="small-muted">No timeline
events for this batch yet.</div>'; return; }

```

```

let html = "";

rows.forEach(r=>{

  // pick color/class per role

  let roleClass = 'role-farmer';

  if (r.actor_role === 'collector') roleClass = 'role-collector';

  else if (r.actor_role === 'processor') roleClass = 'role-processor';

  else if (r.actor_role === 'supplier') roleClass = 'role-supplier';

  else if (r.actor_role === 'export') roleClass = 'role-export';


  let actorName = r.actor_full_name || (r.actor_id ? (r.actor_role + ' #' + r.actor_id) : '');

  let header = `<div class="meta"><div><span class="role-badge
${roleClass}">${escapeHtml(r.actor_role)}</span> <strong style="margin-
left:10px">${escapeHtml(actorName)}</strong></div><div class="small-
muted">${escapeHtml(r.timestamp)}</div></div>`;

  html += `<div class="card-t">

    ${header}

    <div style="margin-top:8px;font-weight:600">${escapeHtml(r.status)}</div>

    <div style="margin-top:6px;color:var(--muted)">${escapeHtml(r.notes || 'No
notes')}</div>

    ${r.location ? `<div style="margin-top:8px;font-size:13px;color:var(--muted)">Location:
${escapeHtml(r.location)}</div>` : ""}

  </div>`;

});

wrap.innerHTML = html;

}

// ----- Add log -----

```

```

qs('#addLogForm').addEventListener('submit', function(e){
    e.preventDefault();

    if (!selectedBatchId && !qs('#log_batch_id').value) return showToast('Select a batch first
(use "Add Log" on a batch row)', false);

    const fd = new FormData(this);
    fd.append('action','add_log');

    // ensure batch id set
    if (!fd.get('batch_id') || fd.get('batch_id') === '') fd.set('batch_id', selectedBatchId);
    fetch(location.pathname, {method:'POST', body: fd}).then(r=>r.json()).then(res=>{
        if(!res.success) return showToast(res.message || 'Add log failed', false);
        showToast(res.message || 'Added');
        // clear and refresh timeline/logs
        this.reset();
        loadLogs(1);
        if (fd.get('batch_id')) viewTimeline(fd.get('batch_id'));
    }).catch(()=> showToast('Server error', false));
});

```

```

function selectBatchForLog(batchId, batchCode){
    selectedBatchId = batchId;
    qs('#log_batch_id').value = batchId;
    showToast('Selected batch: ' + batchCode);
    viewTimeline(batchId);
}

```

```

// ----- Delete log -----

```

```

function deleteLog(id){
  if (!confirm('Delete this log?')) return;
  const fd = new FormData();
  fd.append('action','delete_log');
  fd.append('id', id);
  fetch(location.pathname, {method:'POST', body: fd}).then(r=>r.json()).then(res=>{
    if(!res.success) return showToast(res.message || 'Delete failed', false);
    showToast(res.message || 'Deleted');
    loadLogs(1);
    if (selectedBatchId) viewTimeline(selectedBatchId);
  }).catch(()=> showToast('Server error', false));
}

// ----- Batch suggestions (autocomplete) -----
const batchSearch = qs('#batchSearch'), batchSuggestions = qs('#batchSuggestions');
let batchTimer = null;
batchSearch.addEventListener('input', function(){
  const q = this.value.trim();
  if (q.length < 1) { batchSuggestions.style.display = 'none'; return; }
  clearTimeout(batchTimer);
  batchTimer = setTimeout(()=> {
    const fd = new FormData(); fd.append('action','batch_suggestions'); fd.append('q', q);
    fetch(location.pathname, {method:'POST', body: fd}).then(r=>r.json()).then(res=>{
      if (!res.success) { batchSuggestions.style.display='none'; return; }
      const rows = res.data;
      if (!rows || rows.length===0) { batchSuggestions.style.display='none'; return; }

```

```
    batchSuggestions.innerHTML = rows.map(r => `<p onclick="chooseBatchSug(${r.id},
'${escapeJs(r.batch_code)}')"><strong>${escapeHtml(r.batch_code)}</strong><br><small
style="color:#6b7a86">${escapeHtml(r.crop_type)} —
${escapeHtml(r.farmer_name)}</small></p>`).join("");
```

```
    batchSuggestions.style.display = 'block';
```

```
});
```

```
}, 200);
```

```
});
```

```
function chooseBatchSug(id, code){
```

```
    batchSearch.value = code;
```

```
    batchSuggestions.style.display = 'none';
```

```
    loadBatches(1, code);
```

```
}
```

```
// ----- helpers -----
```

```
function escapeJs(s){ return String(s||'').replace(/'/g,"\\'").replace(/"/g,"\\"); }
```

```
// filters / refresh
```

```
qs('#refreshBtn').addEventListener('click', ()=> { batchSearch.value=""; loadBatches(1);
loadLogs(1); qs('#timelineWrap').innerHTML = '<div class="small-muted">No batch
selected. Click "View Timeline" on a batch row to load details.</div>'; });
```

```
qs('#applyLogsFilter').addEventListener('click', ()=> loadLogs(1));
```

```
batchSearch.addEventListener('keydown', function(e){ if (e.key === 'Enter') { loadBatches(1,
batchSearch.value.trim()); batchSuggestions.style.display='none'; } });
```

```
// initial load
```

```
loadBatches(1);
```

```
loadLogs(1);
```

```
document.addEventListener('click', function(e){  
  if (!qs('#batchSearch').contains(e.target)) batchSuggestions.style.display='none';  
});  
</script>
```

```
</body>
```

```
</html>
```